Script and AI Search Analytics Capture Plan

Date: April 27, 2026
User requested freshness target: April 26, 2026
Target app: `https://pstest.morloksmaze.com`
Hosting: Netlify Functions
Database: Supabase Postgres
Primary goal: Capture useful analytics from scripts, script searches, documentation searches, AI assistant searches, and AI provider usage without storing raw script content, raw prompts, or raw search text by default.

Sources Used

- Supabase Row Level Security docs: https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase query optimization docs: https://supabase.com/docs/guides/database/query-optimization
- Supabase pg_stat_statements docs: https://supabase.com/docs/guides/database/extensions/pg_stat_statements
- Supabase Cron docs: https://supabase.com/docs/guides/cron
- Netlify Scheduled Functions docs: https://docs.netlify.com/functions/scheduled-functions/
- Netlify Functions environment variable docs: https://docs.netlify.com/functions/environment-variables/
- OpenAI Usage and Costs API docs: https://platform.openai.com/docs/api-reference/usage/costs?api-mode=responses&lang=curl
- OpenAI token and billing usage reference: https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them
- Snowplow tracking plans and event specifications: https://docs.snowplow.io/docs/fundamentals/tracking-plans/
- Algolia click and conversion event guidance: https://www.algolia.com/doc/guides/sending-events

Local Findings

The codebase already has partial analytics support:

Existing surface	Location	What it captures
`ai_metrics` hosted schema	`supabase/migrations/20260424_hosted_schema.sql`	AI endpoint, model, tokens, cost, latency, success
Hosted AI metric recorder	`netlify/functions/api.ts`	Records OpenAI/Anthropic calls for hosted routes
Local AI analytics middleware	`src/backend/src/middleware/aiAnalytics.ts`	Sequelize model and aggregation helpers, but route usage is incomplete
`scripts`	`src/db/schema.sql` and hosted schema	Script owner/category/public/version/hash timestamps
`script_analysis`	`src/db/schema.sql` and hosted schema	Security, quality, risk, suggestions, command details
`chat_history`	`src/db/schema.sql` and hosted schema	Conversation records and optional embedding field
Script execution fields/logs	`src/db/schema.sql`	`execution_count`, `average_execution_time`, `last_executed_at`, `execution_logs` in local schema

Current gaps:

- No general `analytics_events` or `user_events` table exists.
- Script searches are not recorded as analytics events.
- Documentation searches are not recorded as analytics events.
- Search results are not linked to later result clicks, script views, downloads, copies, analyses, or saves.
- AI searches/chat are tracked for provider cost in hosted `ai_metrics`, but not as product behavior events.
- Local Express AI metrics are partly unwired; hosted Netlify should remain the first target.
- Direct prompt/query/content capture would be high risk, so default analytics should use hashes and buckets.

Design Principles

1. Keep raw events append-only. Do not mutate analytics facts after insert except for retention cleanup.
2. Keep `ai_metrics` focused on provider cost, token, latency, success, and model accounting.

3. Add a separate `analytics_events` table for product behavior.
4. Use server-side capture first because it is more trustworthy and keeps secrets off the browser.
5. Add frontend capture only for actions the server cannot see, such as copy, local download click, filter change, or result click before navigation.
6. Never store raw script content, raw prompts, raw search queries, raw transcripts, full IP addresses, or full user agents by default.
7. Use `search_id` to connect a search event to later click/conversion events, following the same pattern used by search analytics systems that tie clicks back to a query.
8. Enforce tenant privacy with Supabase RLS and indexed `user_id` filters.
9. Roll up dashboards from aggregates, not from unbounded raw scans.

Recommended Data Model

1. Keep `ai_metrics`

Keep the current `ai_metrics` table for provider billing and performance.

Recommended additive columns:

```
alter table public.ai_metrics
  add column if not exists request_id uuid,
  add column if not exists session_id uuid,
  add column if not exists provider text,
  add column if not exists cached_tokens integer not null default 0,
  add column if not exists fallback_used boolean not null default false;

create index if not exists idx_ai_metrics_user_created
  on public.ai_metrics (user_id, created_at desc);

create index if not exists idx_ai_metrics_request_id
  on public.ai_metrics (request_id)
  where request_id is not null;
```

Reason: local cost estimates should come from response `usage`, but finance reconciliation should compare against OpenAI's Costs endpoint because OpenAI documents that Costs is the finance-grade source.

2. Add `analytics_events`

Use one narrow event ledger for product behavior:

```
create table if not exists public.analytics_events (
  id uuid primary key default gen_random_uuid(),
  occurred_at timestamptz not null default now(),
  received_at timestamptz not null default now(),
  user_id uuid references public.app_profiles(id) on delete set null,
  session_id uuid,
  request_id uuid,
  search_id uuid,
  event_name text not null,
  event_version smallint not null default 1,
  source text not null check (source in ('frontend', 'netlify', 'backend', 'syste
  route text,
  script_id bigint references public.scripts(id) on delete set null,
  category_id bigint references public.categories(id) on delete set null,
  success boolean,
  latency_ms integer,
  result_count integer,
  rank_position integer,
  query_hash text,
  properties jsonb not null default '{}'::jsonb,
  privacy_tier text not null default 'standard'
    check (privacy_tier in ('standard', 'diagnostic_opt_in', 'system')),
  check (jsonb_typeof(properties) = 'object'),
  check (pg_column_size(properties) <= 4096)
);
```

Indexes:

```
create index if not exists idx_analytics_events_user_time
  on public.analytics_events (user_id, occurred_at desc);

create index if not exists idx_analytics_events_name_time
  on public.analytics_events (event_name, occurred_at desc);

create index if not exists idx_analytics_events_script_time
  on public.analytics_events (script_id, occurred_at desc)
  where script_id is not null;

create index if not exists idx_analytics_events_search_id
  on public.analytics_events (search_id)
  where search_id is not null;

create index if not exists idx_analytics_events_request_id
  on public.analytics_events (request_id)
  where request_id is not null;
```

Add a GIN index only when dashboard filters prove it is needed:

```
create index if not exists idx_analytics_events_properties_gin
  on public.analytics_events using gin (properties jsonb_path_ops);
```

Partitioning: start with this single table and composite indexes. Move to monthly range partitioning on `occurred_at` before the table reaches tens of millions of rows or if retention deletes become expensive. The Supabase/Postgres guidance supports partitioning for very large time-series tables, but it adds operational complexity.

3. Optional Search Impressions Table

Only add this if you need result-position analytics beyond clicked items:

```
create table if not exists public.analytics_search_impressions (
  id bigserial primary key,
  search_id uuid not null,
  occurred_at timestamptz not null default now(),
  user_id uuid references public.app_profiles(id) on delete set null,
  result_type text not null check (result_type in ('script', 'documentation', 'ai
  result_id text not null,
  rank_position integer not null,
  score_bucket text,
  clicked boolean not null default false
);
```

Phase 1 can skip this. For most dashboards, `search_performed` plus `result_clicked` is enough.

RLS and Privacy

Enable RLS:

```
alter table public.analytics_events enable row level security;

create policy "analytics events readable by owner"
on public.analytics_events
for select
to authenticated
using ((select auth.uid()) = user_id);

create policy "analytics events insertable by owner"
on public.analytics_events
for insert
to authenticated
with check ((select auth.uid()) = user_id);
```

Important notes:

- Netlify server-side inserts should use validated user identity from `requireUser(req)`.
- If direct browser inserts are allowed, route them through `POST /api/analytics/events` anyway so event names and properties can be allowlisted.
- Admin analytics should be served by Netlify endpoints after an admin check, not by broad client-side RLS policies.
- Use `(select auth.uid())` in policies and indexes on `user_id` to follow Supabase RLS performance guidance.

- Store `query_hash = hmac_sha256(normalized_query, ANALYTICS_HASH_SECRET)`, not the raw query.
- Store coarse buckets: `query_length_bucket`, `result_count_bucket`, `line_count_bucket`, `file_size_bucket`.
- Keep full prompts, transcript text, raw script content, and raw search queries out of `properties`.

Required Netlify environment variable:

```
ANALYTICS_HASH_SECRET=<random 32+ byte secret in Netlify Functions scope>
```

Netlify docs state function runtime environment variables must be available to the Functions scope, and should be read with `process.env` or the Netlify runtime env APIs.

Event Taxonomy

Use stable names and versioned properties. This follows the tracking-plan style recommended by analytics platforms like Snowplow: document event meaning, trigger, source, and valid properties before implementing.

Script Lifecycle

Event	Trigger	Required properties
`script_uploaded`	`/api/scripts` create or upload	`file_size_bucket`, `line_count_bucket`, `is_public`, `duplicate_detected`
`script_viewed`	Script detail route loads	`view_source`
`script_updated`	Script update succeeds	`changed_fields`, `version`
`script_deleted`	Script delete succeeds	`delete_mode`
`script_downloaded`	User downloads `.ps1`, `.psm1`, or zip	`format`, `source`
`script_copied`	User copies generated or saved script	`source`, `line_count_bucket`
`script_analyzed`	Static or AI analysis finishes	`analysis_type`, `security_score_bucket`, `risk_score_bucket`, `quality_score_bucket`
`script_execute_attempted`	Hosted execute endpoint is hit	`hosted_supported`, `result`
`script_similar_requested`	Similar scripts requested	`threshold_bucket`, `result_count_bucket`

Search

Event	Trigger	Required properties
`script_search_performed`	`/api/scripts/search`	`query_hash`, `query_length_bucket`, `filters`, `result_count_bucket`, `search_mode`
`documentation_search_performed`	`/api/documentation/search`	`query_hash`, `source_filter`, `result_count_bucket`
`chat_history_search_performed`	`/api/chat/search`	`query_hash`, `result_count_bucket`
`ai_example_search_performed`	`/api/scripts/examples` or `/api/ai-agent/examples`	`query_hash`, `result_count_bucket`, `model`
`search_result_clicked`	User clicks any search result	`search_type`, `result_type`, `rank_position`, `target_id`
`search_result_converted`	Search leads to analyze/save/download/copy	`conversion_type`, `search_type`, `rank_position`

Each search response should include:

```
{
  "search_id": "1a96a4a6-70c9-4a9e-97ec-dbcf8cf49f51",
  "results": []
}
```

The frontend must pass that `search_id` into click/conversion events. This is the same core idea as Algolia's `queryID`: a later click is only useful for search analytics if it can be connected back to the originating search.

AI

Event	Trigger	Required properties
`ai_request_started`	Chat, stream, generate, explain, analyze starts	`endpoint`, `agent_type`, `request_id`
`ai_request_succeeded`	Provider response succeeds	`endpoint`, `provider`, `model`, `request_id`, `metric_id`
`ai_request_failed`	Provider response fails	`endpoint`, `provider`, `model`, `error_code`, `request_id`
`ai_fallback_used`	OpenAI falls back to Anthropic or local fallback	`from_provider`, `to_provider`, `endpoint`
`ai_output_saved_as_script`	User saves generated script	`source_endpoint`, `line_count_bucket`, `script_id`
`ai_voice_synthesized`	TTS route succeeds/fails	`voice`, `speed_bucket`, `success`
`ai_voice_recognized`	STT route succeeds/fails	`duration_bucket`, `success`

`ai_metrics` should hold token/cost numbers. `analytics_events` should hold product behavior and link to the same `request_id`.

Instrumentation Points

Netlify Function First

Add a reusable helper in `netlify/functions/api.ts` or a shared module:

```
type AnalyticsEventInput = {
  userId?: string;
  sessionId?: string;
  requestId?: string;
  searchId?: string;
  eventName: string;
  source: 'frontend' | 'netlify' | 'backend' | 'system';
  route?: string;
  scriptId?: number;
  categoryId?: number;
  success?: boolean;
  latencyMs?: number;
  resultCount?: number;
  rankPosition?: number;
  query?: string;
  properties?: Record<string, unknown>;
};

async function recordAnalyticsEvent(event: AnalyticsEventInput): Promise<void> {
  // Validate eventName against an allowlist.
  // Normalize and HMAC query into query_hash.
  // Drop raw prompt, content, transcript, and query fields.
  // Insert best-effort; never block the user path on analytics failure.
}
```

Call it from:

- `handleScripts` on GET/POST for list/create.
- `handleScriptSearch` for `script_search_performed`.
- `handleScriptById` for view, similar, analyze, execute attempted.
- Hosted AI routes: please, generate, explain, examples, analysis assistant.
- `handleChat` and `handleChatStream`.
- `handleDocumentation` for documentation search.
- Voice routes for voice product events, keeping transcripts out.

Frontend Gaps

Add a small frontend service:

```
analytics.track('search_result_clicked', {
  search_id,
  search_type: 'script',
  result_type: 'script',
  target_id: script.id,
  rank_position: index + 1,
});
```

Use it only where the server cannot know the action:

- Search result click.
- Copy script.
- Download script.
- Filter changed.
- Dashboard card opened.
- Command palette navigation.

Batch browser events:

```
navigator.sendBeacon('/api/analytics/events', blob)
```

Fallback to `fetch(..., { keepalive: true })` when `sendBeacon` is unavailable. Keep batches small and validate again on the server.

Aggregations and Dashboards

Raw Drill-Down

Keep raw `analytics_events` for 90 days. Use it for debugging and event-level validation.

Daily Rollups

Use SQL views first. Promote to materialized views or rollup tables when query latency needs it.

Recommended rollups:

```
create materialized view if not exists public.analytics_daily_user as
select
  date_trunc('day', occurred_at)::date as day,
  user_id,
  count(*) as events,
  count(distinct session_id) as sessions,
  count(*) filter (where event_name = 'script_uploaded') as uploads,
  count(*) filter (where event_name = 'script_analyzed') as analyses,
  count(*) filter (where event_name like '%search_performed') as searches,
  count(*) filter (where success = false) as failures,
  avg(latency_ms) filter (where latency_ms is not null) as avg_latency_ms
from public.analytics_events
group by 1, 2;
```

```
create materialized view if not exists public.analytics_daily_search as
select
  date_trunc('day', occurred_at)::date as day,
  event_name,
  user_id,
  count(*) as searches,
  count(*) filter (where coalesce(result_count, 0) = 0) as zero_result_searches,
  avg(result_count) as avg_results
from public.analytics_events
where event_name in (
  'script_search_performed',
  'documentation_search_performed',
  'chat_history_search_performed',
  'ai_example_search_performed'
)
group by 1, 2, 3;
```

```
create materialized view if not exists public.analytics_daily_script as
select
  date_trunc('day', occurred_at)::date as day,
  script_id,
  count(*) filter (where event_name = 'script_viewed') as views,
  count(*) filter (where event_name = 'script_downloaded') as downloads,
  count(*) filter (where event_name = 'script_copied') as copies,
  count(*) filter (where event_name = 'script_analyzed') as analyses,
  count(*) filter (where event_name = 'script_execute_attempted') as execute_atte
from public.analytics_events
where script_id is not null
group by 1, 2;
```

AI cost dashboards should continue to aggregate from `ai_metrics`, with optional joins on `request_id` into `analytics_events`.

Scheduled Jobs

Use Supabase Cron for database-local cleanup and materialized view refresh:

```
select cron.schedule(
  'refresh-analytics-hourly',
  '15 * * * *',
  $$refresh materialized view concurrently public.analytics_daily_user;$$
);
```

Use Netlify Scheduled Functions for external reconciliation:

- Daily OpenAI Usage/Costs API pull.
- Compare OpenAI organization/project spend to local `ai_metrics`.
- Alert if local estimated cost differs from provider cost by more than a threshold.

Netlify scheduled functions run on published deploys and have a 30 second execution limit, so keep them narrow. For longer DB-only jobs, prefer Supabase Cron.

Dashboard Metrics

Script Analytics

- Active users and active sessions.

- Uploads by day, category, public/private.
- Duplicate upload rate.
- Analysis runs by type.
- Security/risk/quality score distribution.
- Top scripts by views, downloads, copies, analyses.
- Hosted execution attempts, currently useful because hosted execution returns unavailable.
- Category growth and category drift.
- Search-to-script-view and search-to-analysis conversion.

Search Analytics

- Script search volume.
- Documentation search volume.
- AI examples search volume.
- Zero-result rate.
- Repeat query hashes.
- Result click-through rate.
- Average clicked rank.
- Searches that lead to script save/analyze/download.
- Filters that cause zero results.

AI Analytics

- Requests by endpoint, model, provider.
- Tokens and cost by endpoint/model/provider.
- p95 latency and error rate.
- Fallback rate.
- Cost per successful script analysis.
- Cost per saved generated script.
- Budget alerts.
- Provider reconciliation delta from OpenAI Costs API.

Implementation Phases

Phase 1: Tracking Contract and Schema

Files:

- New migration in `supabase/migrations/`.
- Optional matching migration in `src/db/migrations/` for local parity.
- New docs file for event taxonomy if this plan becomes code.

Tasks:

1. Add `analytics_events`.
2. Add indexes and RLS policies.
3. Add `request_id`, `session_id`, `provider`, `cached_tokens`, and `fallback_used` to `ai_metrics`.
4. Add `ANALYTICS_HASH_SECRET` to Netlify Functions environment.
5. Define event allowlist and property allowlist in code.

Verify:

- RLS blocks cross-user reads.
- Insert rejects invalid event names.
- Raw prompts, raw queries, and script content are not inserted.

Phase 2: Server-Side Capture

Files:

- `netlify/functions/api.ts`
- Possible shared helper under `netlify/functions/_shared/analytics.ts`

Tasks:

1. Add `recordAnalyticsEvent`.
2. Add `request_id` to AI call flow and `ai_metrics`.
3. Return `search_id` from script/documentation/example search routes.
4. Record searches, zero results, script creates/views/analyses, AI starts/success/failure, and hosted execution attempts.
5. Make analytics writes best-effort and non-blocking.

Verify:

- Existing AI and script routes still return the same user-facing payloads plus additive IDs.
- Analytics write failure does not fail the product request.
- Search response includes `search_id`.

Phase 3: Frontend UI Events

Files:

- New `src/frontend/src/services/analytics.ts`.
- Search pages, script cards, download/copy controls, AI example viewer.

Tasks:

1. Add `analytics.track` with batching.
2. Capture result clicks with `search_id`.
3. Capture copy/download/export actions.
4. Capture filter changes only after debounce.

Verify:

- Browser never sends raw script content or raw prompt text.
- Events include auth token and fail quietly on network errors.

Phase 4: Dashboards and API

Files:

- `netlify/functions/api.ts`
- `src/frontend/src/pages/Analytics.tsx`

Tasks:

1. Add `GET /api/analytics/product/summary`.
2. Add `GET /api/analytics/search/summary`.
3. Extend existing AI analytics page with provider/cost/search conversion panels.
4. Add admin-only cross-user views.

Verify:

- Normal users only see their analytics.
- Admin sees aggregate analytics through admin-checked Netlify route.
- Queries use indexed filters on `user_id` and date range.

Phase 5: Rollups, Retention, Reconciliation

Tasks:

1. Add Supabase Cron job for materialized view refresh.
2. Add retention job for raw event cleanup.
3. Add Netlify scheduled function for OpenAI Costs API reconciliation.
4. Add alert thresholds for daily/monthly budget and local/provider cost mismatch.

Retention defaults:

Dataset	Retention
Raw `analytics_events`	90 days
Raw `ai_metrics`	180 days
Daily rollups	24 months
Diagnostic opt-in payloads	7 days

Test Plan

Database:

- Migration applies cleanly.
- RLS permits owner reads and blocks another user.
- Indexes support `user_id + occurred_at`, `event_name + occurred_at`, `script_id + occurred_at`, and `search_id`.
- `EXPLAIN` dashboard queries before shipping.

Netlify:

- `POST /api/analytics/events` accepts valid batches and rejects unknown event names.
- Script search writes one `script_search_performed` event.
- Search route returns `search_id`.
- Script result click writes `search_result_clicked`.
- AI chat/generate/explain/analyze writes both `analytics_events` and `ai_metrics`.
- Analytics write failure is logged but does not fail the original route.

Frontend:

- Unit tests prove no raw prompt/script/query is sent in analytics events.
- Click/download/copy events include `search_id` when available.
- Beacon fallback works.

Hosted:

- Deploy to Netlify.
- Confirm `/api/health`.
- Run authenticated probes for search, chat, script analyze, download/copy event.
- Confirm rows in Supabase for the admin account.
- Confirm `/api/analytics/ai/summary` and new product/search summaries return 200.

Acceptance Criteria

- Product analytics and AI cost analytics are separate but linkable by `request_id`.
- Script searches have `search_id`.
- Clicks/conversions can be attributed to the originating search.
- Zero-result search rate is measurable.
- AI cost by endpoint/model/provider is measurable.
- No raw script content, raw prompts, raw transcripts, or raw search strings are stored in default analytics.
- RLS prevents users from reading other users' raw events.
- Admin dashboards use server-side authorization.
- Dashboard queries do not scan unbounded raw events.

Recommended First Implementation Slice

Implement this first:

1. Add `analytics_events` migration with RLS and indexes.
2. Add `recordAnalyticsEvent` to Netlify.
3. Instrument only:
4. `/api/scripts/search`
5. `/api/documentation/search`
6. `/api/chat`
7. `/api/chat/stream`
8. `/api/scripts/analyze/assistant`
9. `/api/scripts/generate`
10. `/api/scripts/explain`
11. Return `search_id` from search routes.
12. Add a small frontend analytics service for `search_result_clicked`, `script_downloaded`, and `script_copied`.
13. Add `GET /api/analytics/search/summary`.

This slice gives immediate visibility into the most important behavior: what users search for, whether searches return results, what results they click, what AI routes cost, and whether AI output turns into saved/analyzed scripts.

Generated 2026-04-29 14:08 UTC